

Compiladores

Aula 17: Ambientes de execução e geração de código

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Endereçamentos
- Código de destino
- CALL e RET

Endereçamentos

- O modelo conceitual de um espaço de endereçamento inclui sistemas reais que podem organizar a memória de outras formas.

CÓDIGO	instruções executáveis	disponível durante a execução
ÁREA ESTÁTICA	globais e dados de endereço fixável	normalmente, toda a execução
PILHA	frames e dados associados a chamadas	aloca/libera com chamadas e retornos
HEAP (MONTE)	objetos de duração dinâmica	vida não ajustada a uma única ativação

Endereçamentos

- Variáveis locais pertencem a uma ativação e podem ocupar um slot de frame.

```
int somaAte(int n) {
  int acumulado = 0;
  int i;
  for (i = 1; i <= n; i = i + 1)
    acumulado = acumulado + i;
  return acumulado;
}
```

Durante a chamada

n, acumulado e i são valores próprios desta execução de somaAte.

Ao retornar

O espaço deixa de representar variáveis vivas daquela chamada; pode ser reutilizado.

Fim da vida ≠ apagamento dos bits.

Endereçamentos

- O frame guarda o contexto de uma ativação.
- A pilha combina com chamadas aninhadas porque, em geral, elas retornam em ordem inversa à abertura.
- Parâmetros: *valores recebidos*
- Retorno: *onde continuar no chamador*
- Estado salvo: *registradores exigidos pela convenção*
- Locais / temporários: *dados da ativação*
- Elo de controle: *frame do chamador*
- Elo de acesso*: *ambiente léxico envolvente*

* Elo de acesso é necessário em certos modelos com funções aninhadas; não é requisito mínimo do MINIC desta disciplina.

Endereçamentos

- Uma chamada recursiva cria outro frame.
- Cada ativação preserva seu próprio parâmetro e o ponto de continuação.

```
int fatorial(int n) {
  if (n <= 1) return 1;
  return n * fatorial(n - 1);
}
```



Retorno em cascata: fatorial(1) → 1; fatorial(2) → 2 × 1; fatorial(3) → 3 × 2.

Endereçamentos

- Escopo determina a declaração enquanto o ambiente localiza o valor.
- São dimensões relacionadas, mas não equivalentes: vínculo de nome é estático; vida do objeto é dinâmica.

```
int limite = 10;
int conta(int x) {
  int resultado;
  resultado = x + limite;
  return resultado;
}
```

x e resultado

Locais à ativação corrente; tipicamente encontrados no frame.

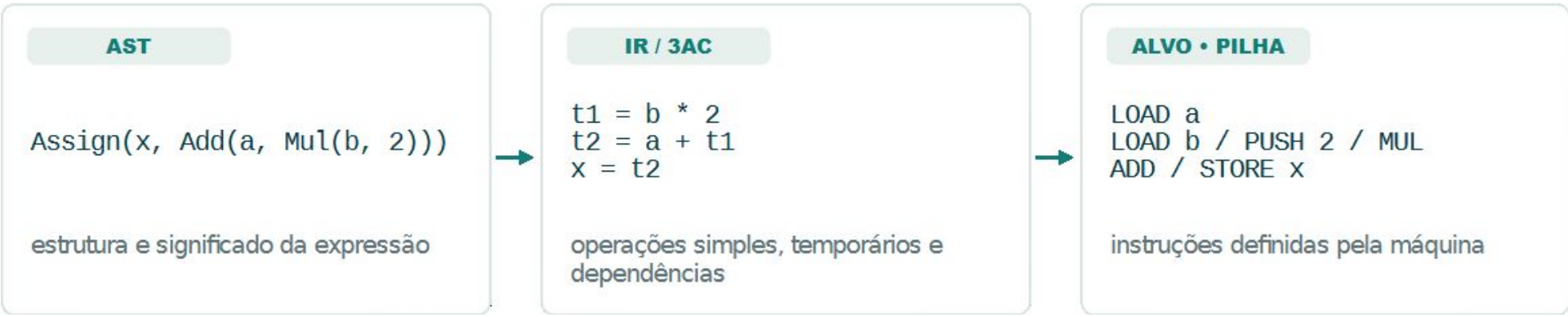
limite

Global: associação e endereço podem ser resolvidos sem buscar um frame ancestral.

Elo de controle ≠ elo de acesso

Código de destino

- A sequência de passos após o parser: AST ⟶ IR ⟶ código de destino
- Cada representação responde a uma necessidade diferente, sem alterar o comportamento observável.



O gerador pressupõe IR consistente e deve preservar tipos, controle, efeitos e comportamento observável.

Código de destino

- A escolha depende da máquina-alvo e do custo relativo entre instruções e acessos à memória.

IR

```
t1 = a + b
t2 = t1 * c
x  = t2
```

Alvo esquemático

```
LOAD R0, a
ADD  R0, R0, b
MUL  R0, R0, c
STORE x, R0
```

Seleção de instruções

Uma operação da IR pode virar uma ou várias instruções do alvo.

Alocação e atribuição

Decidir quais valores vivem em registradores e quais registradores concretos usar.

Spill

Quando os registradores não bastam, valores são guardados em slots de memória.

O exemplo de alvo é didático: operandos e formatos variam por arquitetura.

Código de destino

- Máquina de pilha: cada operação consome e produz valores.
- A pilha de operandos guarda resultados temporários; a pilha de chamadas guarda ativações.

```
x = a + b * 2;

LOAD a
LOAD b
PUSH 2
MUL
ADD
STORE x
```

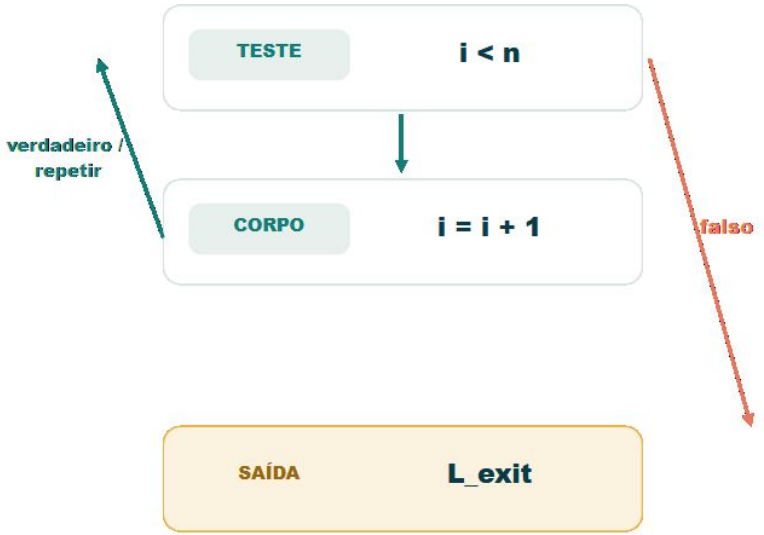
Instrução	Pilha após executar
LOAD a	[a]
LOAD b	[a, b]
PUSH 2	[a, b, 2]
MUL	[a, b×2]
ADD	[a+b×2]
STORE x	[]

Código de destino

- Rótulos tornam o fluxo explícito.
- O gerador precisa manter cada caminho de execução ligado ao destino correto.

```
while (i < n){
   i = i + 1;
}
```

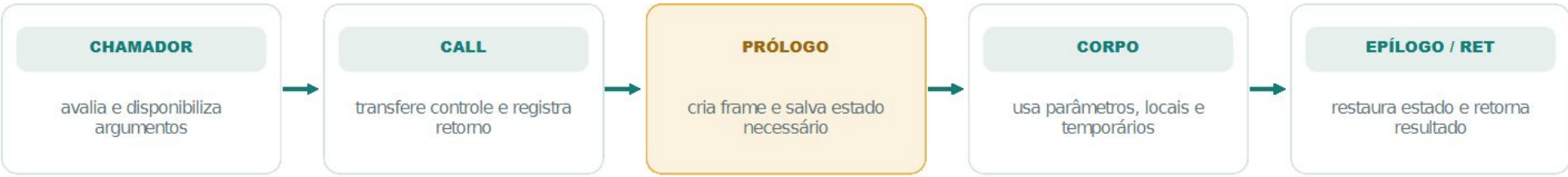
```
L_test:
t1 = i < n
IF_FALSE t1 GOTO L_exit
i = i + 1
GOTO L_test
L_exit:
```



Em for, continue deve levar ao passo correto; break, à saída do laço.

CALL e RET

- CALL e RET precisam obedecer ao mesmo contrato.
- Prólogo e epílogo são decisões da convenção da VM ou da ABI, não sequências universais.





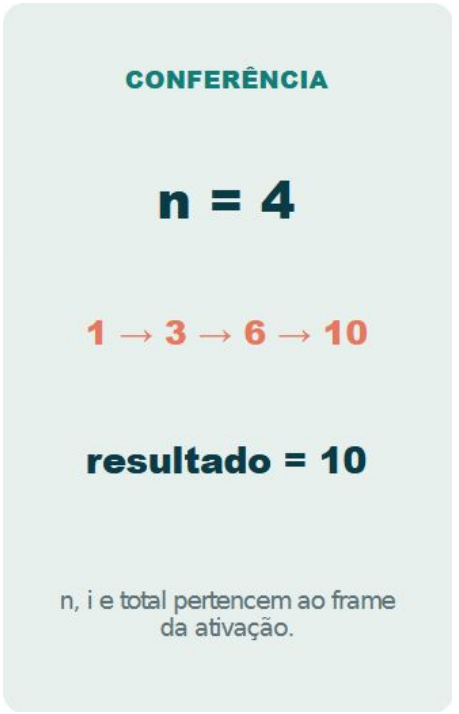
Em recursão, cada CALL cria um novo frame, mesmo para a mesma função.

CALL e RET

- Exemplo de uma função vista como MINIC, IR e execução.

```
int somaAte(int n) {
  int i = 1, total = 0;
  while (i <= n) {
    total = total + i;
    i = i + 1;
  }
  return total;
}
```

```
FUNCTION somaAte(n)
i = 1; total = 0
L_test:
t1 = i <= n
IF_FALSE t1 GOTO L_exit
t2 = total + i
total = t2
i = i + 1
GOTO L_test
L_exit: RETURN total
```



CALL e RET

- Verifique o modelo de memória, o fluxo e o contrato de chamada antes de atribuir significado ao alvo.

"Todo local é global."	→ Locais e parâmetros são próprios de cada ativação.
"Recursão reutiliza um frame."	→ Cada chamada ativa exige estado e retorno independentes.
"Heap é obrigatório no MINIC."	→ A especificação mínima não define alocação dinâmica explícita.
"Qualquer ordem economiza registrador."	→ Reordene só se preservar efeitos, curto-circuito e comportamento definido.